

1. Static/dynamic operation**03A1**

我们知道，视其是否会改变数据集的内容及结构，对数据结构的操作可分为**静态**、**动态**两种。

- 然而如果数据集中各元素的**逻辑**结构与**物理**结构未必一致，此时的静态、动态操作又当如何区分？
- 你能举出这类数据结构的一例吗？

2. Call-by-rank/position**03A2**

- 秩与位置之间有何联系与区别？
- 通过重载操作符“`[]`”可令列表模仿向量的“循序访问”方式，但我们为何**不建议**经常使用这种方式？

3. List[-k]**03A2**

在一个由单向链表实现、也不记录总体长度的列表`L`中，为找到倒数第`k`个节点`L[-k]`，有以下两种方法：

- 先经过一次遍历，统计出`n = L.size`；然后再次遍历，找到节点`L[n-k]`。
- 使用间距为`k`的两个位置指针`p`和`q`，同步地遍历列表，待到`q`抵达`tail`时，返回`p`。

不难看出，两种方法的渐进复杂度同阶，但实际的运行时间却可能差异很大。

试通过实验就此做一测试和统计，并做出解释。

4. Distance Between List Nodes**03A2**

任给同一列表中的两个节点，我们需要通过如下接口，得到它们之间的距离：

```
template <typename T> Rank distance( LNP<T> p, LNP<T> q );
```

当然，时间复杂度必须为 $\mathcal{O}(d)$ ，其中 d 就是返回的距离。

- 如果**保证**`p`在`q`之前，试借助本节重载的减号操作符（`ListNode<T>::operator-`）实现`distance()`；
- 如果二者的前后次序**未知**呢？

5. Sentinel**03B[1-4]**

- 引入哨兵`head`、`tail`之后，列表主要接口的代码逻辑在哪些方面更为**精简**了？
- 运行**时间**方面，这些接口各有多大的改进？
- 针对本节介绍的各种列表接口，验证借助哨兵的确可以正确处理各种**边界**情况（比如空表）。

6. insert()/remove()**03B1**

本节实现的节点插入、删除算法中，都有若干对`pred`、`succ`的调整。这些调整的**次序**是否可以调换？

7. copy()**03B2**

- 试在构造时调转方向，改用`insertAsFirst()`而非`insertAsLast()`来实现`List::copy()`，并依然可以保证新列表中的所有节点，能够保持原先的**相对次序**；
- 如此调整之后，算法的**时间**、**空间**性能有何变化？

8. dedup()**03B3**

- 该算法的循环不变性为“始终维护一个无重的**前缀**”，如果改为**后缀**，又当如何实现该算法？效率如何？
- 该算法在前缀中一旦发现与节点`p`**相等**的`q`，便随即删除`q`——如果反过来删除`p`，代码需如何调整？
- 自行设计若干测例，验证该算法的正确性，尤其能正确处置各种**边界**情况；
- 特别地，试验证：该算法对`find()`**首次**调用时，查找区间总是**为空**；尽管如此，`find()`亦能**正确**处理；

e) 试按照其它思路，设计和实现这个接口，并做实测对比。

9. traverse(): Basic

03B4

- a) 对照Vector::traverse()和List::traverse()接口，理解通过**函数指针**或**函数对象**的实现方式；
- b) 在实例代码包中，找到并阅读increase_Elem.h、decrease_Elem.h、double_Elem.h、half_Elem.h等源文件，了解如何通过**函数对象**来定义对单个元素的基本操作；
- c) 阅读increase_vector.h、double_vector.h等源文件，理解如何将上述自定义的基本操作**顺序地施加至**向量中的每个元素，以实现向量的遍历；
- d) 阅读decrease_list.h、half_list.h等源文件，理解如何将上述自定义的基本操作**顺序地施加至**列表中的每个元素，以实现列表遍历；
- e) 模仿上述套路定义自己的基本操作，并相应地实现对向量、列表的遍历。

10. traverse(): Advanced

03B4

- a) 阅读sum_Elem.h，理解如何通过**函数对象**的构造函数，以参数形式传入、传出全局性信息；
- b) 阅读sum_list.h、sum_vector.h，了解如何通过**遍历接口**来计算序列的总和（作为校验码）；
- c) 阅读inv_Elem.h，理解如何通过**函数对象**的构造函数，以参数形式传入、传出全局性信息；
- d) 阅读inv_list.h、inv_vector.h，了解如何通过**遍历接口**来判定序列是否有序；
- e) 模仿上述套路定义自己的（涉及全局性信息的）基本操作，并相应地实现对向量、列表的遍历。

11. Find & Search By Traversal

03B4

不难看出，无论无序列表还是有序列表，其查找算法的实质过程，无非也是对列表的遍历。
试照此理解，基于List::traverse()接口来实现List::find()及List::search()算法。

12. uniquify()

03C

- a) 该算法起始处，为何需要对平凡的列表（_size < 2）做**特判**？
- b) 你觉得可以如何改写，使得这段代码的逻辑更为**简洁**？
- c) 自行设计若干测例，验证该算法的正确性，尤其能正确处置各种**边界**情况；
- d) 试按照其它思路，设计和实现这个接口，并做实测对比。

13. search(): failure cases

03C

- a) 试确认，在查找**失败**时，该算法将返回查找区间的直接前驱x；
- b) 试确认，对于任何元素e，在调用“insert(search(h, e, t), e)”之后，区间(h,t)**依然**有序；
- c) 如果只是**单纯**的查找，如何根据search(h, e, t)的返回值，**判断**区间(h,t)内是否**存在**元素e呢？

14. search(): efficiency

03C

- a) 试确认，即便列表已经**有序**，运用目前已讲授的技术仍不能有效地降低search()接口的复杂度；
- b) 这一现象背后的根本**原因**，你认为是什么？

15. Edge Cases

03D

- a) 试验证：即便区间(h,t)内**只有**一个节点，本节的List::selectionSort()算法也能正常终止；
- b) 试验证：即便区间内**没有**任何节点，该算法也能正常终止。

16. selectionSort() by movePred()**03D**

本节实现的List::selectionSort()算法, 直接通过movePred()接口将当前最大的节点插入有序的后缀。

- 试通过实践验证: 也可先经remove()摘除x, 再通过insert()来实现**同样**的效果;
- 为何我们还是倾向于使用讲义中的方法?

17. selectionSort() by exch()**03D**

在选择排序中对无序前缀中的最大元素, 既可以采用**平移法**将其转入有序后缀, 也可以采用**交换法**。

- 试增加ListNode::exch(BinNodePosi<T> x)接口, 在 $O(1)$ 时间内将当前节点与x**交换**;
- 试实现这个接口, 并与List::selectionSort()原版本就性能做一实测对比。

18. getMax()**03D**

- 试验证: getMax()算法的确可以从列表区间中找到**最大**元素, 且在有多个时会选取**最靠后者**;
- 试验证: 区间内的每一组相等元素, 至多只会引发最大节点记录 (x) 的一次更新;
- 如果将扫描的次序由**自后向前**改为**自前向后**呢?
- 如果**自前向后**扫描, 在最坏情况下最大节点记录需要更新多少次? 试给出这样的一个实例。

19. Vector::selectionSort**03D**

- 试基于向量实现**就地的选择排序**算法, 要求与列表一样, 虽然仍需做 $O(n^2)$ 次**比较**, 但仅需做 $O(n)$ 次**移动**;
- 为保证移动操作不超过 $O(n)$ 次, 在将每个元素归入有序后缀时, 你应采用**交换法**还是**平移法**?
- 进一步地, 为保证**稳定性**, 应采用**交换法**还是**平移法**?
- 从实例代码包中找到已经实现了的Vector::selectionSort(), 就上述方面, 与你的实现做一比较。

20. Swap vs. Cycle**03E**

- 在一个序列中交换一对元素, 序列中的哪些轮换会有**变化**?
- 变化可能有几种**情况**? 试逐一列举并说明。

21. Expected #Cycle By Exchanges**03E**

考查对长度为 n 的随机序列做选择排序, 约定采用**交换法**。

按照我们的惯例, 在每一步迭代中, 无序前缀里的**最大元素**与**末元素**分别记作M和T。

- 试证明: 若M和T不是同一节点, 则在二者交换之前, 它们应当**同属**一个轮换, 且在其中**紧邻**;
- 试证明: 二者交换之后, 该轮换的长度**缩减一个单位**, 而M**自成一个**轮换;
- 不难看出, 当M和T系同一节点 (M业已就位) 时, 对二者的交换操作是**多余的**。

试证明: 如果输入序列包含 c 个轮换, 那么这种情况在其选择排序过程中会**恰好**出现 c 次;

- 试证明: 对于任何一步迭代, 若当时无序前缀的长度为 r , 则发生这种情况的概率为 $1/r$;
- 试证明: 采用**交换法**的选择排序过程中, 期望地会出现 $O(\log n)$ 次多余的**交换**;
- 试证明: 在长度为 n 的随机序列中, **期望地**含有 $O(\log n)$ 个轮换。

22. Expected #Cycle By Moves**03E**

- 若选择排序改用**平移法**, 是否也有类似上一题中c)的结论? 亦即, 对于包含 c 个轮换的输入序列, 选择排序中恰好出现 c 次无效的**移动**? 如果是, 试给出证明; 否则请举出反例;

e')是否也有类似与上一题e)的结论? 亦即, 在选择排序过程中, 期望地会出现 $\mathcal{O}(\log n)$ 次多余的**移动**? 如果是, 试给出证明; 否则请举出反例;

23. insertionSort() vs. selectionSort()

03F

- a) 我们知道插入排序属于**在线**算法, 你所知道的其他排序算法中还有哪些也是?
- b) 在玩纸牌游戏时若希望手中的牌始终顺序排列, 你在抓牌过程中为何会采用**插入**排序而非其它算法?
- c) 我们知道插入排序属于**输入敏感**算法, 你所知道的其他排序算法中还有哪些也是?
- d) 我们知道**选择**排序既非在线的, 也没有最好情况, 那么反过来, 它相对于**插入**排序有何优势?

24. Vector::insertSort

03F

- a) 试基于向量实现**就地的插入排序**算法;
- b) 你能否做到与列表一样,, 虽然仍需做 $\mathcal{O}(n^2)$ 次**比较**, 但仅需做 $\mathcal{O}(n)$ 次**移动**?
如果能, 如何做到? 否则, 为什么不能?

25. insertionSort() by move()

03F

本节实现的List::insertSort()算法, 直接通过moveSucc()接口将节点x插入有序的前缀。

- a) 试通过实践验证: 也可先经remove()摘除x, 再通过insert()来实现**同样**的效果;
- b) 为何我们还是倾向于使用讲义中的方法?

26. Best/average case of insertionSort()

03F

- a) 我们知道, 当输入序列业已有序时插入排序只需运行 $\mathcal{O}(n)$ 时间, 这种最好情况出现的概率有多大?
- b) 新元素被插入有序前缀后, 有可能其实原地未动。期望而言, 在整个排序过程中这种情况会出现多少次?
- c) 平均而言, 插入排序需要运行多少时间?

27. search() in insertionSort()

03F

我们知道, 插入排序算法在插入每个新元素之前, 都要在有序前缀中查找确定合适的位置。

- a) 既然此处被查找的前缀始终都**有序**, 为何不将**顺序查找**改作**二分查找**?
- b) 如果必须改为**二分查找**, 存储有序前缀的数据结构需要哪些接口? 各接口的效率需要有多高?

28. Complexity of 2-way merge

03G

- a) 如果待归并的列表长度不等甚至相差悬殊, 二路归并算法的运行时间还能保持线性吗?
- b) 一般地, 若二者的长度分别为 n 和 m , 运行时间应是多少?

29. 2-way merge by move()

03G

本节实现的List::merge()算法, 直接通过movePred()接口将节点x移到p之前。

- a) 试通过实践验证: 也可先经remove()摘除x, 再通过insert()将其归入最终列表;
- b) 为何我们还是倾向于使用讲义中的方法?

30. Vector/List::mergeSort()

03G

基于向量或列表来实现归并排序, 各有什么优势与不足?

31. Vector/List::mergeSort()

03G

我们知道, 无论是基于向量或列表来实现归并排序, 只要如讲义那样始终以中点来切分子任务 (子序列),

就能保证总体的时间复杂度为 $O(n \log n)$ 。

- a) 如果改为总是按 $2/3 : 1/3$ 的比例来切分呢?
- b) 改用 $15/16 : 1/16$ 的比例呢?
- c) 一般地, 改用 $\lambda : 1 - \lambda$ 的比例呢?
- d) 如果总是按 $n - 1 : 1$ 来切分, 那么mergeSort()相当于**退化**成了哪种排序算法?

32. #inversion

03H

- a) 试确认, 一个序列中所含的**逆序对**总数可能多达 $\Omega(n^2)$, 并给出这样的实例;
- b) 就**数学期望**而言, 一个长度为 n 的序列中会含有多少个逆序对?
- c) 根据上一结论, 插入排序算法的**平均**时间复杂度应是多少?

33. #inversion

03H

设一个长度为 $2n$ 的向量, 用02D节的bubblesort()提前终止版, 经过 n 轮扫描交换便完成了排序。

- a) 该向量**最少**、**最多**可能包含多少个逆序对?
- b) 若 $n = 15$, 试分别给出一个对应的整数向量实例。

34. Inversion & Exchange

03H

- a) 在序列中交换一对逆序元素, 逆序对的**总数**会如何变化? 可能有哪几种**情况**?
- b) 如果可以选择, 在排序过程中我们为何倾向于优先交换相距**更远**的逆序对?
- c) 试证明: 一个序列中的逆序对无论多少, 我们都只需 $O(n)$ 次**交换**便可使之有序;
- d) 你所知的排序算法中, 哪些可以保证所做的交换操作**不超过** $O(n)$ 次? 哪些不能保证?

35. Counting inversions

03H

- a) 试确认, 借助归并排序必能在 $O(n \log n)$ 时间内**统计**出任一序列中的逆序对总数;
- b) 你还有什么其它办法也可以完成这一统计?
- c) 有没有**更加高效**的办法?

36. Cursor list

03XA

- a) 在什么情况下, 我们需要以游标模式来实现 (或者更准确地, 来模拟) 列表?
- b) 对讲义中的实例, 理解这种结构的原理, 推敲其运作过程, 并尝试实现该结构。

37. Java Vector/List

03XB

- a) 了解Java语言中interface、implements等关键词的含义与作用;
- b) 在示例代码包中找到用Java描述和实现的数据结构及算法;
- c) 阅读其中Vector和List的代码, 与C++的实现做一对比;
- d) 运行自带的测试程序, 理解测试过程及结果。

38. Python List

03XC

- a) 在示例代码包中找到用Python描述和实现的数据结构及算法;
- b) 阅读源文件reverse.py, 理解并**对比**其中那两个reverse()算法;
- c) 了解这里所用的测试方法, 运行并观察统计结果。